

Tripper

**Project submitted in partial fulfilment for the award of
the degree of Masters of Computer Application**

**Submitted by
Debmalya De Sarkar (1839010)**

**Under the Guidance of
Dr. Chinmay Mishra
Associate professor**



**KIIT DEEMED TO BE UNIVERSITY
SCHOOL OF COMPUTER APPLICATION
APRIL 2020.**

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled **TRIPPER** submitted to **School of Computer Application, KIIT Deemed To Be University** in partial fulfilment of the requirement for the award of the degree of **MASTER OF COMPUTER APPLICATIONS (MCA)**, is an authentic and original work carried out by **Mr. Debmalya De Sarkar** with Roll no. 1839010 and Regd. No. 18327662053 under the guidance of **Dr Chinmay Mishra**.

The matter embodied in this project is genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfilment of the requirements of any course of study.

Debmalya De Sarkar
.....

Signature of the Student:

Date:

.....

Signature of the Guide

Date:.....

School of Computer Application
KIIT University, Bhubaneswar
CERTIFICATE

This is to certify that the project work entitled **Tripper** Submitted by **Debmalya De Sarkar** bearing roll no. **1839010**, is an authentic and original work.

Signature

(Internal Examiner)

Date.....

Signature

(External Examiner)

Date.....

DECLARATION

I Debmalya De Sarkar, roll no. 1839010 do hereby declare that the project report entitled **Tripper** submitted to **School of Computer Application, KIIT Deemed To Be University, Bhubaneswar** for the award of the degree of **MASTER OF COMPUTER APPLICATION (MCA)** , is an authentic and original work carried out by me from 3rd JAN 2020 to 26st May 2020 at Organization name under the guidance of **Dr Chinmay Mishra**.

Debmalya De Sarkar
.....

Signature of the student

Date.....

ACKNOWLEDGEMENT

I have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals. I would like to extend my sincere thanks to all of them. I am highly indebted to **Dr Chinmay Mishra** and **Dr. Veena Goswami** for their guidance and constant supervision as well as for providing necessary information regarding the project & also for their support in completing the project. I would like to express my gratitude towards my parents & member of **KIIT School of Computer Application** for their kind co-operation and encouragement which help me in completion of this project.

Content

1	Introduction	1
	System Analysis	
	● Identification of Need	2
	● Feasibility Study	3
	● Software requirement specifications (SRS)	5
	● Software Engineering Paradigm applied	21
3	System Design	
	● Modularization details	32
	● Data integrity and constraints	37
	● Class diagram	42
	● User Interface Design	43
4	Coding	
	● Standardization of the coding	49
	● Comments and Description:	53
	● Validation checks	65
5	Testing	
	● Functional Testing Test Cases:	68
	● Performance Testing Test Cases	70
	● Security Testing Test Cases	71
	● Usability Testing Test Cases	72
	● Compatibility Testing Test Cases	73

6	System Security measures ● Database/data security:	74
8	Future scope	78
9	Assumptions	80
10	Glossary	81
11	Bibliography	84

Introduction

Trippler, is a cross platform application developed using Flutter. The work of this application is to make a ride or drive not only easier but smarter too and also it helps in keeping track of the vehicle distance to empty with map support and an Emergency feature to locate that person in tricky situation.

The idea of this project was initiated after a thorough study of people who does lots of traveling and cant really maintain a record of their vehicle's distance to empty and they end up in a scenario where the next fuel station is really far away this is a tough situation to handle in an totally unknown place and also they have to notify their family and friends about this scenario, that's tricky too, where they have to inform each and every one by mentioning details of their location.

Now that become easier with this app as it has a SOS feature that keep track of the current location and send an emergency message to the registered no. and also it gives an warning of low fuel, now with a single tap it tasks you to the nearby fuel station with the help of google map. This app currently

The system has been developed using open source technology viz. Flutter and Dart There is a lot of scope for extending the current system to incorporate more features as required.

System Analysis

Identification of Need:

○ Existing System requirement:

In the current scenario people are traveling with approximation of the idea of how far they can travel with their vehicle also, if it is a unknown place it is difficult to keep track of fuel station if they ran out of fuel and in case of emergency they do notify their family and friends individually and which is challenging.

There is some vehicle which shows information like distance to empty. But they don't have the features that this app like SOS, Map support etc.

○ How computerization is useful:

This cross-platform app has been developed to make a ride or drive real easy, where smart phone is enough to keep track of all this information of the vehicle once register in the app, as it also a cross platform so no need to be specific about which smart phone is used.

By this app it is easier to keep track of how much fuel are left and how much distance can be covered and if fuel is low it redirects to nearby fuel station with just a tap. Due to. Due to the high capacity of computerized system, the user can travel with no tension of the vehicle information of tracking.

Feasibility Study:

A feasibility study evaluates the project's potential for success; therefore, perceived objectivity is an important factor in the credibility of the study for potential investors and lending institutions. There are five types of feasibility study—separate areas that a feasibility study examines, described below

○ **Technical Feasibility:**

The technically feasibility study basically center on alternatives for hardware, software and design approach to determine the functional aspects of system.

Trippler is feasible technically, the hardware required to run this application is easily available in today's market. The open source developing tools that is Flutter, Dart, Firebase are used to develop this app, because this is a cross-platform app it can run in iOS and Android, where Android is widely used. And only authorized users are allowed to access the data. Also, this app back end is supported by Google Firebase and Firebase Database where Authentication is handled by firebase and the encryption is done by Google.

○ **Operational Feasibility:**

Operational Feasibility is a measure of how people can work with system. This type of feasibility demands if the system will work when developed and installed.

This is a cross-platform app it is easily available to any of the OS like Android and iOS, and in backend it uses Firebase as the database management which is available 24 x 7 and managed by Google. The database transaction is encrypted by google and Firebase is also highly secure.

In development of an app UI is one of the most essential elements to work with. As this app is developed using Flutter which is well known for its UI development, so this app is made up with simple and user-friendly UI with attractive animations.

○ **Economic Feasibility:**

Tripper is Economically Feasible, because it is developed using open source technology like Flutter, Dart, Firebase which is free of cost. The software and the hardware resources that are used in developing this app are Economic Feasibility too, because the hardware used are now available in every Android and iOS smart phones, which are available in the market

Software requirement specifications

Introduction

The following subsections of the SRS document provide an overview of the entire SRS.

○ **Purpose:**

The purpose of this project is to make traveling not only easy but also smarter, this app takes the vehicle information of the registered user. It provides information of Distance to empty, Distance traveled in a trip and it provide an alert service when the vehicle is in limited fuel and can redirect to the nearby fuels station. This app is capable of supporting user in a travel with SOS service when there is an situation. It is also capable of redirecting to a specific destiny with the help of Google Map service while the app is running in the background doing its work.

○ **Scope:**

This app provides information of the vehicle's trip on the basis of distance travelled by the register user. This information's are Distance to empty, trip, fuel. This app is targeted for the daily commuters and travelers. The above-mentioned information is one of the essential things to keep track on and while riding or driving, in current scenario it become tough to keep track of this information and people end up in tricky situation. To overcome this issue, I came up with this app named as Tripper, this is capable of tracing vehicle traveling information with alert supports like SOS and Low fuel warning with navigation. In can make a travel very much easier in compare to non-computerize calculations as this is a cross platform app it can run in any mobile device supporting either Android or iOS.

- **Benefits:**

This cross-platform app reduce the manual tracking of the vehicle information in a trip so that the user can contrate on road. This app provides all type of support for a trip on vehicle information, also this provide more accurate information rather then manual tracking. as this uses Firebase.

- **Overall Description:**

In Tripper users can register with a secure medium. Then this app is capable of providing information of the user's vehicle in a trip. This then keep track of the data and stored in Firebase database, which is fetched next time to provide correct and more accurate information of the last trip and ongoing. As this app is feature by Firebase network lost is not a issue, as the device get a stable connection then it uploads the data.

○ **Product Perspective:**

➤ **User Interface:**

The User Interface of Tripper consist of the bellow modules.

- A login screen for entering the username, password.
- Sign up form screen for new users.
- Home.
- Dash Board.
- SOS.
- Top Up.
- Calibration.
- Direction.

➤ **Hardware Interface:**

The Hardware Interface of Tripper consist of the bellow modules.

- GPS.
- RAM.
- CPU.
- Display Driver.

○ **Product Functions:**

This section provides details of all major functionalities supported by Tripper.

➤ **Sign up Module:**

■ **Requirement Description:**

- Collect information from user
- Update user information to Firebase Authentication.
- Update vehicle information to Firebase database.

■ **Exception Scenarios:**

- Firebase don't respond.
- If user credential wrong. It will redirect to the Sign-up page with message "can't register with this credential".
- No network services.

■ **Dependencies:**

- SMS service
- Location Service
- Firebase Service

➤ **Login Module:**

■ **Requirement Description:**

- Registered user enters user id and password.
- User clicks on "Login" button.
- System should authenticate and create a valid user session upon successful authentication and redirect user to product search page.

■ **Exception Scenarios:**

- If authentication fails, user will be redirect back to the login page with a message saying "cannot login with this credentials".
- The operation also redirects to the login page if there is any network error.

■ **Dependencies:**

Authentication service of Google firebase.

➤ **Dash Board Module:**

■ **Requirement Description:**

- Provide all information detail of travelling. This module consist of information of Distance to empty and Trip information.
- Fetching data from Firebase.
- Fetching data from Location Service.
- Calculating Distance to empty.
- Calculating total distance traveled in a trip.

■ **Exception Scenarios:**

- Firebase don't respond.
- Network connecting issue.

■ **Dependencies:**

- User should be logged in.

➤ **Map Module:**

■ **Requirement Description:**

- Provide a button to redirect to Google Map App.

■ **Exception Scenarios:**

- Map app not available.

■ **Dependencies:**

- User should be logged in.
- Map app should be pre-installed.
- Logged in user.

➤ **SOS Module:**

■ **Requirement Description:**

- Fetch Emergency Phone number form Firebase.
- Use Phones SMS service to send emergency message.
- Fetching current location from Location Service.

■ **Exception Scenarios:**

- Firebase don't respond.
- Location service doesn't respond.
- No network services.
- No active SMS plans.

■ **Dependencies:**

- SMS service.
- Location Service.
- Firebase Service.
- Logged in user.

➤ **Top Up Module:**

■ **Requirement Description:**

- Update Fuel capacity in Firebase for the vehicle.
- Calculate Distance to empty and update Firebase data.

■ **Exception Scenarios:**

- Firebase connectivity failure.

■ **Dependencies:**

- Firebase Connectivity.
- User logged in.

➤ **Calibration Module:**

■ **Requirement Description:**

- Update Calibrated mileage value to Firebase database.

■ **Exception Scenarios:**

- Firebase connectivity failure.

■ **Dependencies:**

- Firebase connectivity.
- User logged in.

➤ **Logout Module:**

■ **Requirement Description:**

- This module logout the user from the system.
- Firebase is used to keep track of the user detail.
- It redirects to the login page of the app.

■ **Dependencies:**

- Firebase connectivity.
- User should be logged in.

○ **A summary of the major functions that the website will perform:**

- Signing up new users with there vehicle information
- Provide user to login to isolate their vehicle information form other by authenticate users using Firebase Authentication.
- Provide vehicle information while traveling.
- Emergency service with SOS with their current location detail.
- Distance travelled in a trip.
- Direction service using Google Map app.

○ **Apportioning of Requirements:**

There is some requirement that isn't implemented to the current version. The next version of this app will consist of interaction of the vehicles hardware directly to get more precise information the manual calibration and top up will be done automatically by the vehicle's hardware. SOS Service will be able to get information of nearby police station and ambulance service to notify the emergency situation of the user.

FUNCTIONAL REQUIREMENTS:

It deals with the functionalities required from the system which are as follows:

- This app will be allowing user to register them self with there vehicle information.
- This app will be providing information of a trip for a vehicle like distance to empty, trip, direction.
- The Low fuel warning system provide information of there fuel status which is done by a notification which on tap provide near by fuel station information and can navigate to those station.
- The user can update their fuel status via Top up while refueling that will again change the distance to empty.
- This app provides to calibrate its calculation by updating the information of mileage for their vehicle, because on time the mileage fluctuates.
- This app is featured with Emergency service which is SOS, this with an emergency number keep track of the user's location and in such situation on tap can send message with the location information to the emergency number.

NON-FUNCTIONAL REQUIREMENTS:

They are the quality requirements that stipulate how well a software does what it must do.

○ **Performance:**

Tripper is developed using Flutter which is well known for its UI development, this app has a fluent UI which is smooth and fast in response. The backend is supported by Firebase which is one of the best for mobile development platform, the database is managed by Google there is hardly any delay in response. As Firebase is enabled with offline capabilities that is, whenever the application is offline the transactions of data are stored in cache of the device and when the device is established a connection it uploads the data. The authentication is Because of all this the app is high responsive.

○ **Availability:**

This app will be 100% availability for users, because it is an cross-platform native app its updates will improve its bugs which will not at all impact the installed app in any device. The data base will be available 24 x 7 as it is maintained by Google and a minor improvement in the database will not impact the way the installed app or the currently running app will be impacted. The changes will impact in the next version.

○ **Reliability:**

This app is extremely reliable because 98% time it works fine, while testing in it minimum requirement hardware and software configuration for a week. This app is independent of internet once the user is logged in, because Firebase can store offline data which is updated once the internet is connected.

- **Usability:**

While developing an application the most important thing is usability, with that keep in mind, this app is developed with user friendly and attractive UI, and all modules are self-explanatory which make it easy to use app

- **Portability:**

As this is a cross platform app, this can run in any mobile OS, that include iOS and Android, its Flutter doing its job by converting all dart code to native code.

- **Flexibility:**

It is effort required to modify operational program. This app is build using independent module, which any modification will be done it will not impact other module. Once this app is installed any changes will be done using updating this app form the respective stores of the OS. Any new change will not change the old functionality of the app. So this app is highly flexible.

- **Maintainability:**

This app is easy to maintain in a short span of time, while the database is minted using Firebase so no impact will be done in the currently running app just a next update of the app will impact all the changes. The future update will bring new feature to the app and a manure change in database structure which can max take 1 day to import.

- **Installability:**

This app is a cross-platform based app it easy to install as it will be available to stores in the respective OS, and a user can install by single tap, as it goes for installation uninstallation is that easy too just the procedure of the OS are different. The updation of the apps will also be easy as they will be maintained by the market of the respected OS.

- **Reusability:**

This app is highly reusable in to all kind of smart apps for vehicle. The features provided here are very much important to all kind of smart connect app for any vehicle, each module will add a point to any kind of smart connect vehicle app.

Software Engineering Paradigm applied

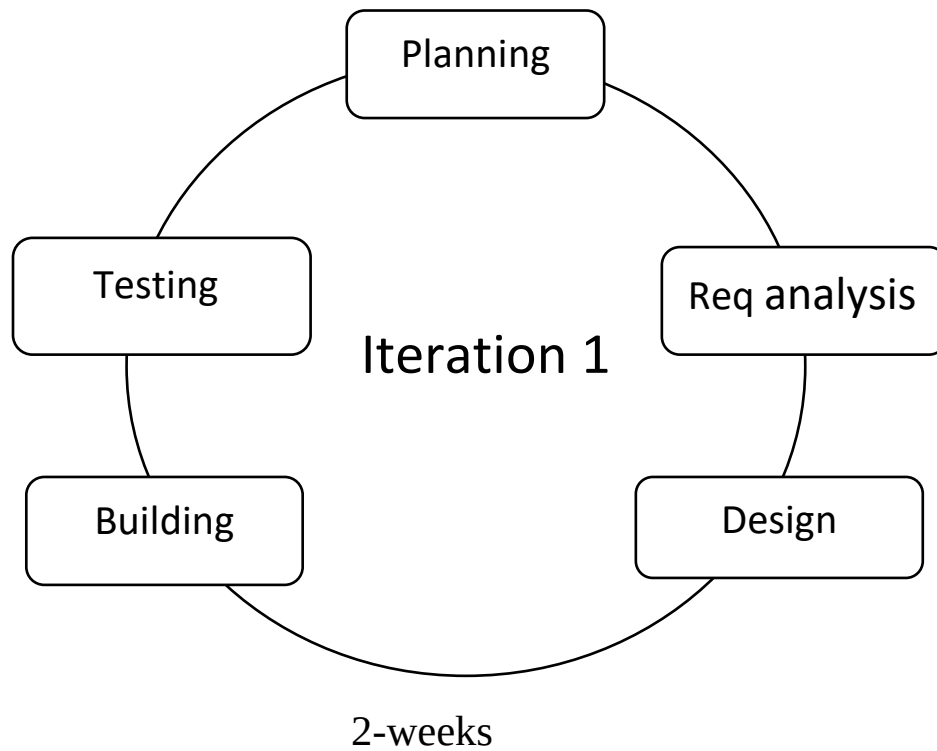
The software engineering paradigm which is also referred to as a software process model or Software Development Life Cycle (SDLC) model is the development strategy that encompasses the process of identifiable stages that a software product undergoes during its lifetime that is a description of software in a systematic and disciplined manner. The life cycle mode used to develop this project is Agile model.

Agile model:

Agile model believes that every project needs to be handled differently and the existing methods need to be tailored to best suit the project requirements. In Agile, the tasks are divided to time boxes (small time frames) to deliver specific features for a release.

Iterative approach is taken and working software build is delivered after each iteration. Each build is incremental in terms of features; the final build holds all the features required by the customer. And every iteration is performed after 2 weeks of span.

○ **Iteration 1**



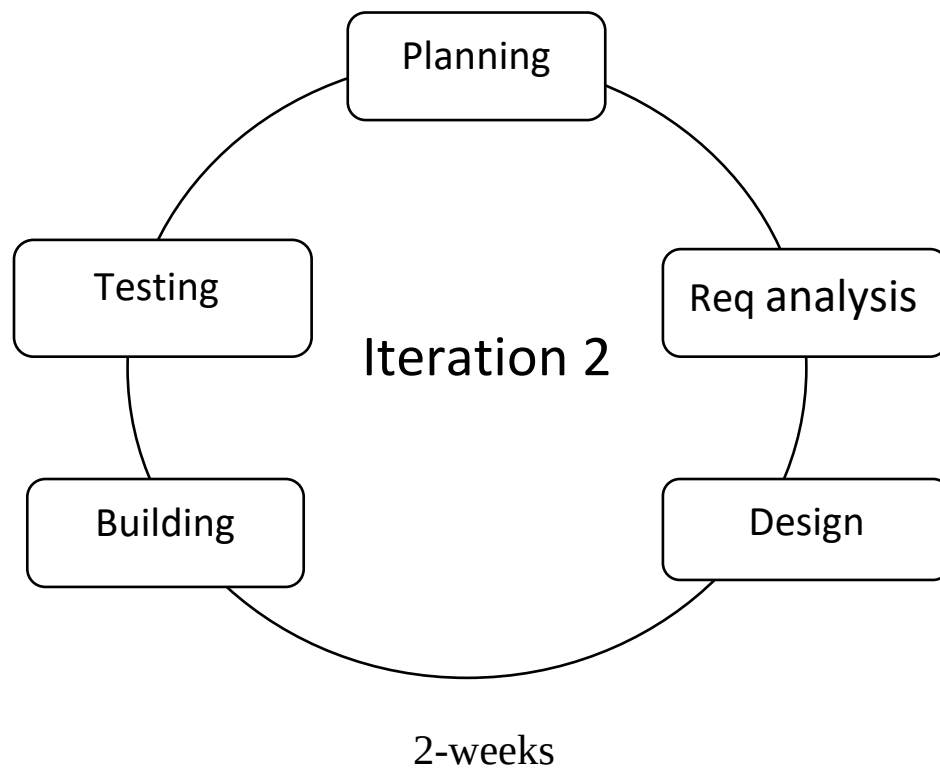
➤ **Bug fixed:**

- The UI has render flow exception which was fixed.
- The Distance calculation was not appropriate.
- Firebase service stream was improved.

➤ **New Functions:**

- Direction was added.

○ **Iteration 2**



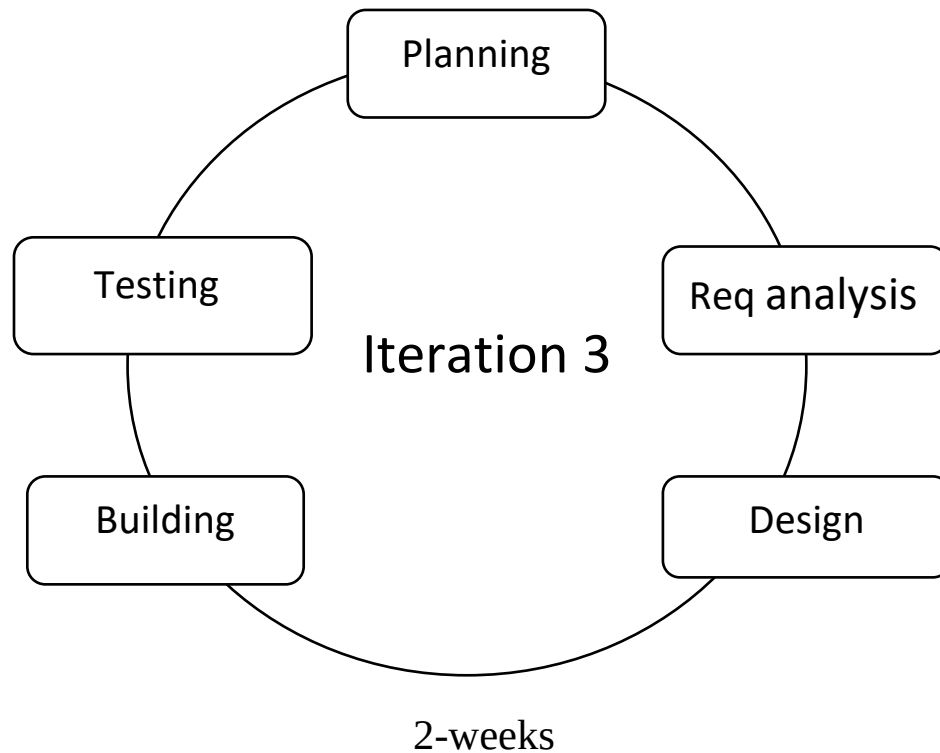
➤ **Bug fixed:**

- The UI has render flow exception for Direction module was fixed.
- Location tracking was fixed.
- Firebase Authentication was improved.

➤ **New Functions:**

- Dark theme was introduced.
- SMS service was added
- Firebase Email registration was enabled

○ **Iteration 3**



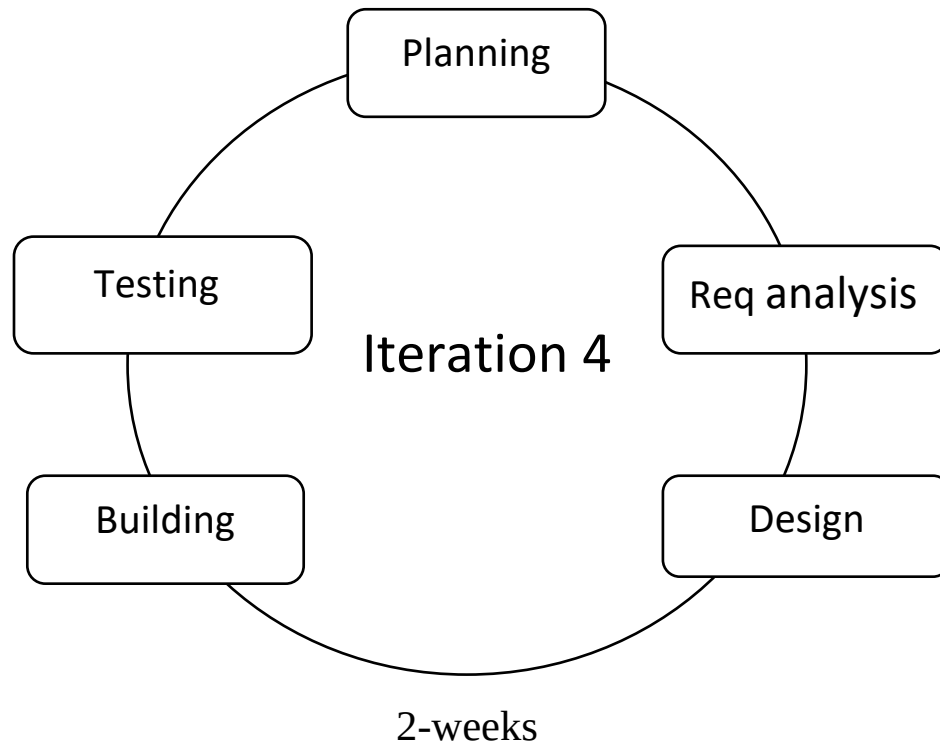
➤ **Bug fixed:**

- SMS service was fixed
- Loading screen was improved
- In Registration module emergency number was added

➤ **New Functions:**

- Google Map Service was enabled.
- New animations were added.
- Strems provider was added

○ **Iteration 4:**



➤ **Bug fixed:**

- Total UI was fixed.
- Location distance calculation was improved.
- Animation improved.

➤ **New Functions:**

- Notification service was added.
- SOS module was added

For every iteration for this project is developed under the following steps:

- **Planning:**

This phase is the very first step of system development life cycle. The objective and goal of system is determined, and the system requirement are considered. An estimate of resources, such as software requirement, hardware requirement that along with the system are proposed. Once the current process requirement is done, then it will process to the next phase which is requirement analysis phase.

- **Requirement Analysis:**

The goal of this phase is to understand the exact requirements of the customer and to document them properly (SRS).

- **Design:**

The goal of this phase is to transform the requirement specification into a structure that is suitable for implementation in some programming language.

- **System Building Phase:**

In this phase will start coding for the system. Dart is used for UI and backend, in Flutter frame work. Google Firebase is used for backend support. The system will be programed on the basis on the outcome from system design. As Flutter frame work is used for the development, it will change to native code for the respective OS to enable cross-platform ability of the app

- **System Testing Phase:**

In this phase, the full testing toward the develop system are performed. This is to ensure that the system has meet the requirements and objectives. Besides that, it is also to ensure that the system is free from error and bug.

Activity Diagram:

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc.

- **Symbols:**

- **Starting point:**

A small filled circle followed by an arrow represents the initial action state or the start point for any activity diagram.



Start Point/Initial State

- **Activity or action state**

An action state represents the non-interruptible action of objects..



Activity

- **Action flow:**

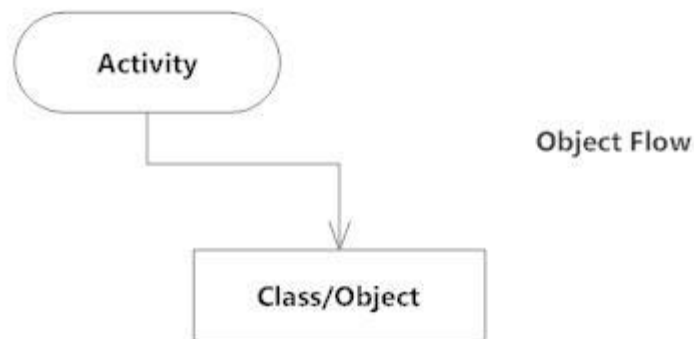
Action flows, also called edges and paths, illustrate the transitions from one action state to another. They are usually drawn with an arrowed line.



Action Flow

- **Object flow:**

Object flow refers to the creation and modification of objects by activities. An object flow arrow from an action to an object means that the action creates or influences the object. An object flow arrow from an object to an action indicates that the action state uses the object.



- **Decision and Branching:**

A diamond represents a decision with alternate paths. When an activity requires a decision prior to moving on to the next activity, add a diamond between the two activities. The outgoing alternates should be labeled with a condition or guard expression. You can also label one of the paths "else."



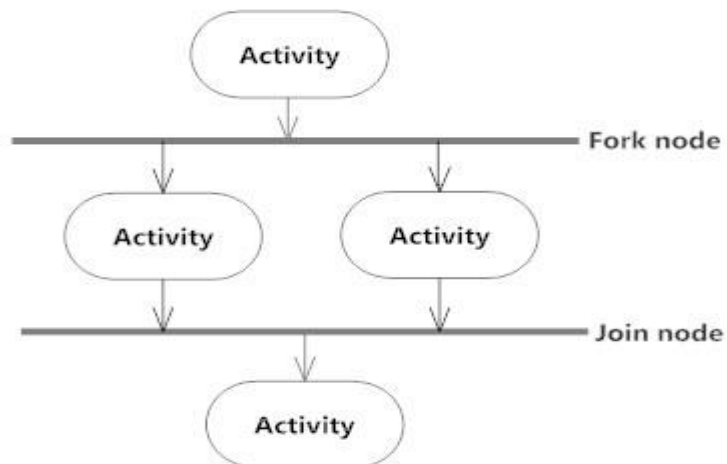
- **Synchronization:**

A fork node is used to split a single incoming flow into multiple concurrent flows. It is represented as a straight, slightly thicker line in an activity diagram.

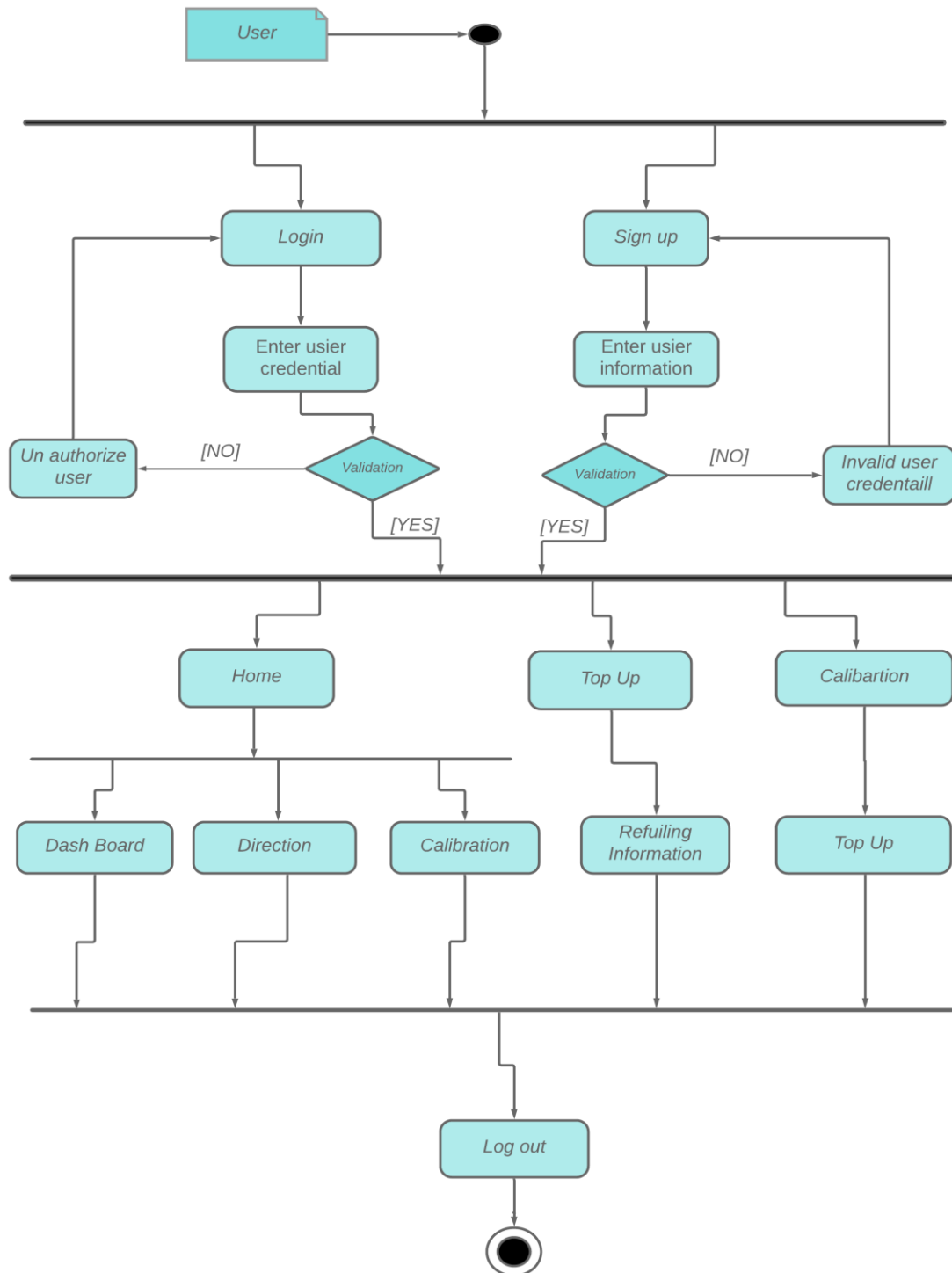
A join node joins multiple concurrent flows back into a single outgoing flow.

A fork and join node used together are often referred to as synchronization.

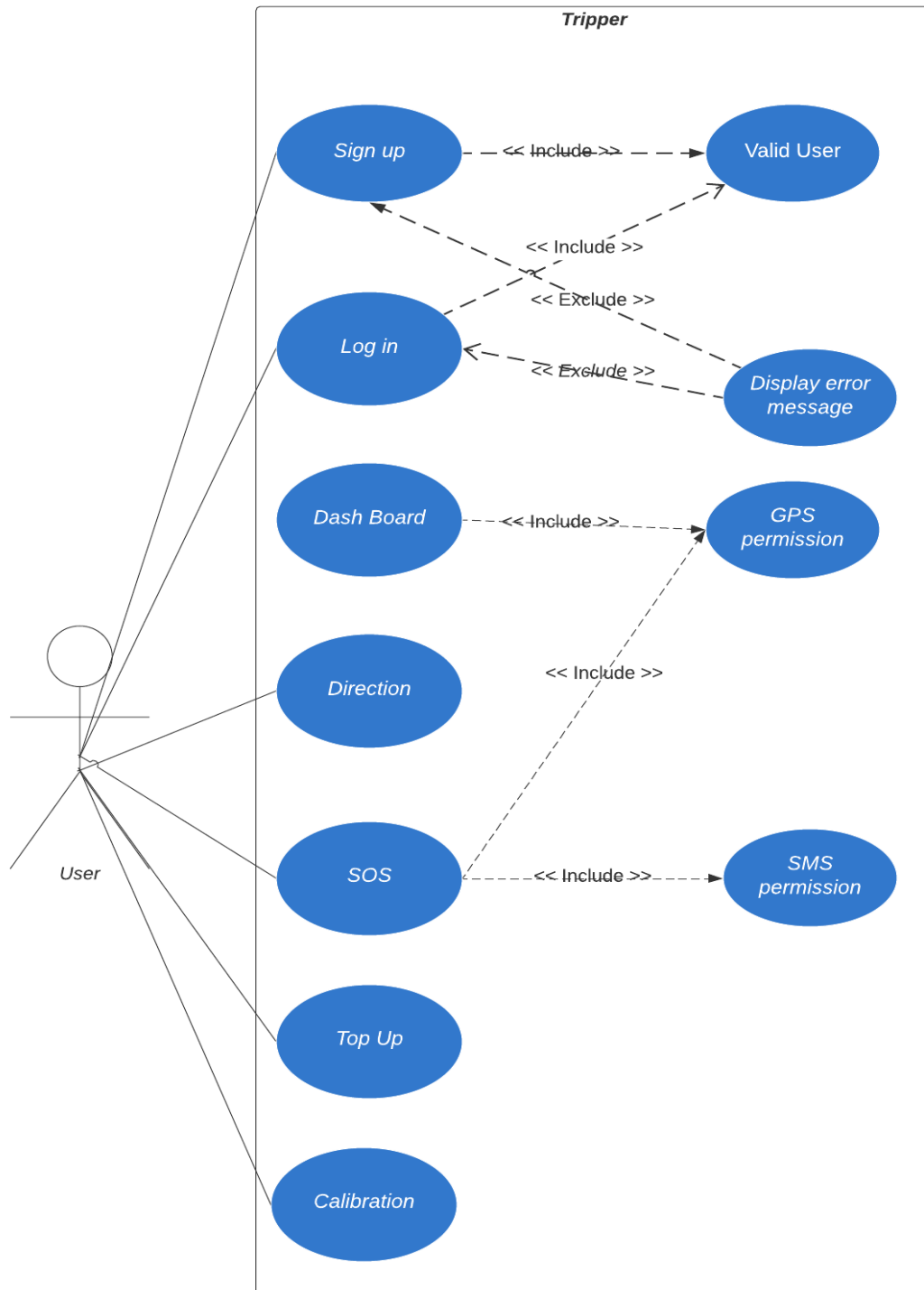
Synchronization



Activity diagram for Tripper



Use case diagram for Tripper:



System Design

Modularization detail:

Structured design partitions the program into small and independent modules. These are organized in top down manner with the details shown in bottom.

Thus, structured design uses an approach called Modularization or decomposition to minimize the complexity and to manage the problem by subdividing it into smaller segments.

○ Login:

Login is the process by which an individual gain access to the system by identifying and authenticating themselves. The authenticated user has unique User ID and Password for to access their personal data. And once the user is login to the system then their user id is retrieved form the database and stored in a session. Once they are logged out their session are destroyed.

Password and user id encryption by Google is used for security, after encrypting then the password is stored in Firebase Auth service. This encryption is done in Firebase and provide encoded password and it doesn't decode, but while the system is checking for the validation it decodes the password internally and check with the password and if it returns 1 then its valid and if 0 then it is invalid. This is one of the secure back-end used, and all the security rules are provided and some are to be provide in near future, to avoid un authorize access. All security rules are tested for accuracy.

Once the user logged in and leave the app without logging out, the next time the app will directly redirect the user to the Home module, this is to make the user to be not bothered to login every time.

- **Signup:**

This module is use for registering new users for this system and register themselves as an authenticate user. This module also checks if the user is already existing, if the user is already there, then it will not accept re registration of the users. While adding new user Firebase auth service is used where the username and password are encrypted by Google, even admin cant see the password in auth for a particular user and the user name saved as hashed which is not readable by human.

Once the user sign up with all valid information it will directly redirect to the Dash board module inside Home module.

- **Home:**

Once the user signs in, they will be redirected to Home module, this module consist of three other module that are Dash Board, Direction, SOS, these three modules are discussed below. This module is basically used for traversal of those three above mentioned modules, that can be achieved by the bottom navigation bar provided inside Home module.

○ **Dash Board:**

This module consist of all important information of the vehicle, that is *Distance to empty, Total trip distance covered and Fuel*, this module require permission of GPS to work, once the user logged in the app will as for location access permission from the user for accessing the GPS of the device to keep track of the user movement, and once the users grant the permission it will start tracking all the traveling information to give a precise result, once tracking is started a user can minimize the app and the app will run in background and there will be no impact on the data shown in this module.

This is the main module of this app, when the app is open this is the module user will find them into. There is background database support form the firebase where it stores all the information, so that next time user logged in then it can show their last travel data's and start calculating from then.

○ **Direction:**

As the name goes, this module consists of a button with on tapped use Google map to redirect users to their destination, as This app runs in background the user can use Google Map for its direction.

As this app is developed wile keeping in mind about the rules and regulation of road, this is enabled with notification service, which notify users about their low fuel situation and on tap this redirect to nearby fuel station of the users.

○ **SOS:**

While traveling emergency situation can appear from nowhere without notifying, so to handle this scenario up to some extent, Tripper is enabled with SOS service, this uses SMS service of the device, at the initial launch of the app it will ask for the SMS service permission.

- **Top up:**

This module consists a direct interaction with vehicle fuel data in Firebase database, this is consisting of an input field with take numeric input and update the current fuel quantity in that Firebase database. This module is helpful while a user has to refuel their vehicle, and they need to update the data with their current fuel quantity.

- **Calibration:**

This module is to sync the current mileage status of the vehicle. This module is developed keeping in mind, that a vehicle with time need service and the mileage impacted with time, if the vehicle is new and fresh, with 1st service the mileage increases and if the vehicle has a fluty engine then with time the mileage decreases.

- **Logging out:**

This module disconnects the user id from the device memory so when ever the app is open next time it will redirect to the login module.

- **Location service:**

This provide accessibility of location with background support, if the app is minimized this will create a threat for the respected OS so that it can keep track of movement while a user us using navigation.

- **SMS service:**

This module provide access to device SMS feature which will enable a user to send message to the register emergency no.

- **Firebase service:**

This module provide dart to interact with the firebase facilities like authentication and Firebase Database.

- **Db Service:**

This module is use for the Firebase database interaction for all other modules.

- **Notification service:**

This module put a local push notification for the a low fuel situation, where as on tap it take the support of URL launcher module to launch map servile.

- **Auth service:**

This module is use for user authentication with Google Firebase.

Data integrity and constraints:

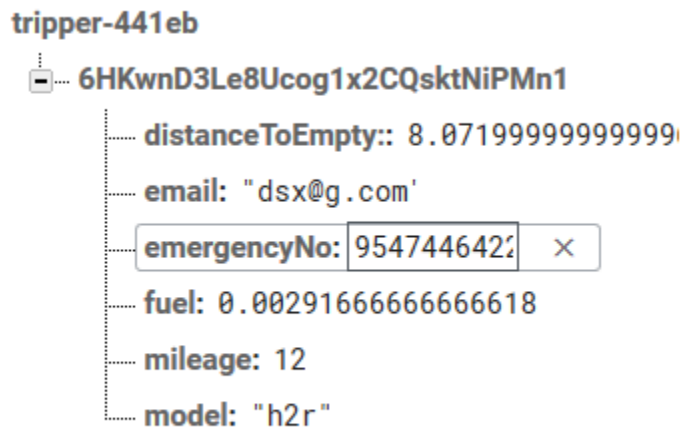
All Firebase Realtime Database data is stored as JSON objects. You can think of the database as a cloud-hosted JSON tree. Unlike a SQL database, there are no tables or records. When you add data to the JSON tree, it becomes a node in the existing JSON structure with an associated key. So the data integrity and constraints are maintained in a different way.

○ Unique Id:

While a user is registered, Firebase Auth generate an unique id for the user, this feature maintains an unique id every time a new user is added, while the collection is created this UID is provided as the name of the collection so that the data can be maintain uniquely for every user.

Identifier	Providers	Created	Signed In	User UID ↑
dsx@g.com		Feb 28, 2020	Apr 13, 2020	6HKwnD3Le8Ucog1x2CQsktNiPM...
d@g.com		Apr 4, 2020	Apr 4, 2020	WqX6pNjScUdCtWJs8kxYdZhS3L...

- **Data Security:**



The data for every user is maintained with the Unique User Id that is been created while the user will register to the app.

- **Security rule:**

```
{  
  /* Visit https://firebase.google.com/docs/database/security to learn more about security rules. */  
  "rules": {  
    ".read": true,  
    ".write": true  
  }  
}
```

There is not much change in security rules in this current version, in the next update security rules will be change to authorize users and require changes will be allowed by the authorize users

○ Data Consistency:

Consistency in database systems refers to the requirement that any given database transaction must change affected data only in allowed ways. Any data written to the database must be valid according to all defined rules.

The data consistency is highly maintained while developing Tripper, while a user is registering to the system the data validation restriction are checked what type of data is passed.

➤ Email:

```
validator: (val) => val.isEmpty ? "Enter a valid email" : null,  
onChanged: (val){  
    setState(() {  
        email = val;  
    });  
},
```

While user is entering email for it can't be null.

➤ Password:

```
validator: (val) => val.isEmpty && val.length < 6 ?  
    "Enter password greater then 6 digit" : null,  
obscureText: true,  
onChanged: (val){  
    setState(() {  
        pwd = val;  
    });  
},
```

While user is entering Password for it can't be null and the length cant be less than 6.

➤ **Emergency no:**

```
keyboardType: TextInputType.number,  
validator: (val) => val.isEmpty && val.length < 10 && val.length > 10 ?  
    "Enter a valid phone no" : null,  
onChanged: (val){  
    setState(() {  
        emergencyNo = val;  
    });  
},
```

While user is entering Emergency no for it can't be null also can't be less than 10 and the input type is number.

➤ **Vehicle Model:**

```
validator: (val) => val.isEmpty && val.length < 2 ?  
    "Enter a valid vehicle model" : null,  
obscureText: false,  
onChanged: (val){  
    setState(() {  
        model = val.toUpperCase();  
    });  
},
```

While user is entering Model, for it can't be null and length can't be less than 2.

➤ **Vehicle Mileage:**

```
keyboardType: TextInputType.numberWithOptions(decimal: true),  
validator: (val) => val.isEmpty && val.length < 1 ? "Enter a valid milage" : null,  
onChanged: (val){  
    setState(() {  
        milage = double.parse(val);  
    });  
},
```

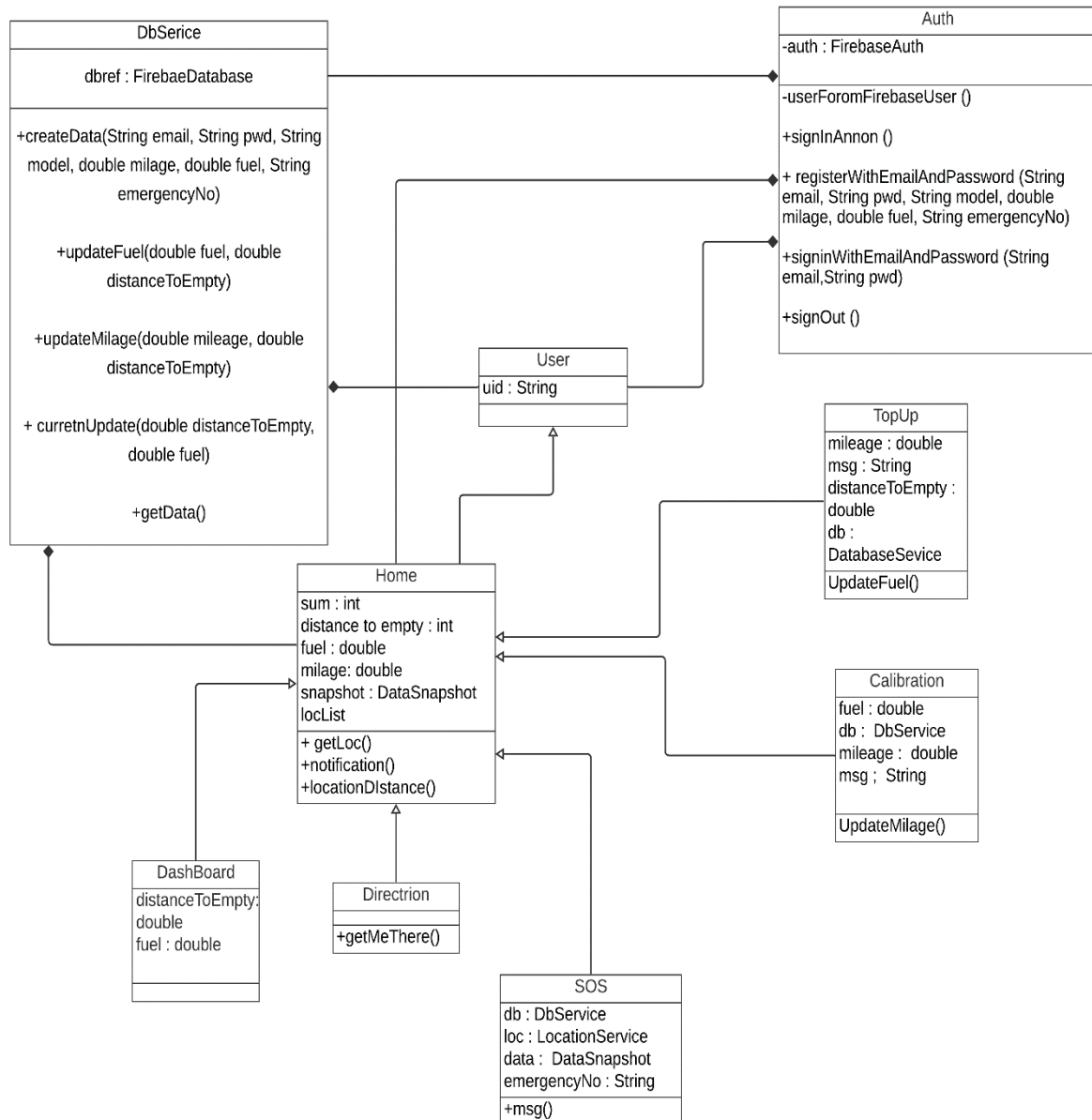
While user is entering Mileage, for it can't be null and length aslo can't be less than 1 and the input type is number.

➤ **Fuel:**

```
keyboardType: TextInputType.numberWithOptions(decimal: true),
validator: (val) => val.length > 3 ? "Enter a valid fuel amount" : null,
onChanged: (val){
  setState(() {
    fuel = double.parse(val);
  });
},
```

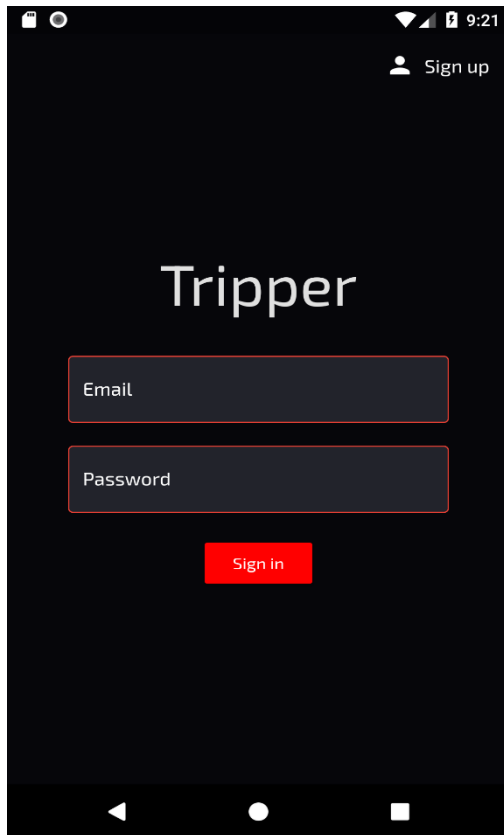
While user is entering Fuel amount, for it can't be null also the length can't be less than 2 and the input type is number.

Class diagram:



User Interface Design:

○ Login screen:



➤ Description:

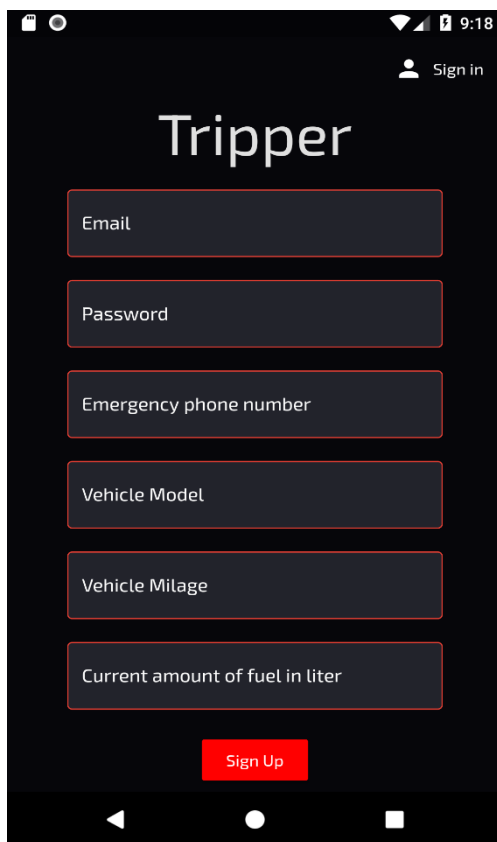
Log in screen enable user to enter their valid credentials to access their authenticate data of the system.

On signing in the user is redirected to the Home screen of the system.

➤ Elements:

- Input box, to enter the username and password for there validation
- Sign in button to transact the data for validation to Firebase Authentication service.
- Sign up button for navigate to Sign up screen.

○ Sign Up Screen:

A screenshot of a mobile application's sign-up screen. The app is titled "Tripper" in a large, white, sans-serif font. Below the title, there are six input fields, each with a light gray border and placeholder text: "Email", "Password", "Emergency phone number", "Vehicle Model", "Vehicle Milage", and "Current amount of fuel in liter". At the bottom of the form is a red button with the text "Sign Up" in white. In the top right corner, there is a "Sign in" link with a user icon. The status bar at the very top shows the time as 9:18 and various system icons. The bottom of the screen shows the standard Android navigation bar with back, home, and recent apps buttons.

➤ Description:

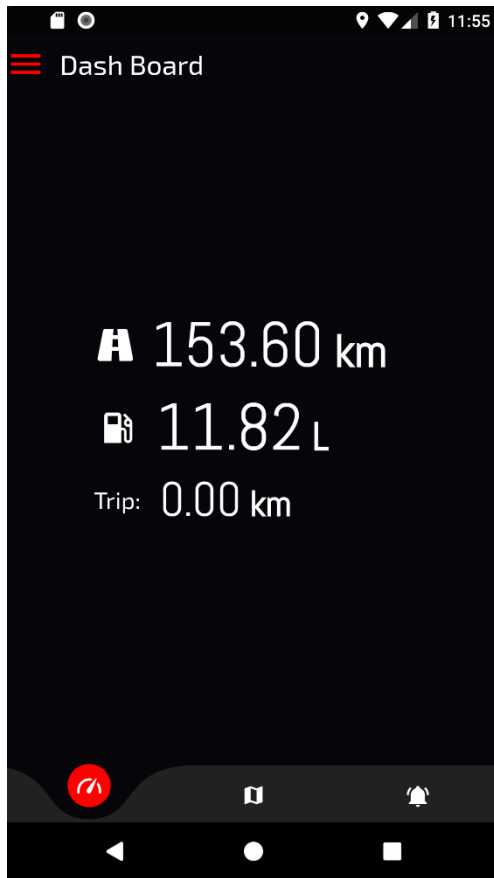
Sign up screen enable user to enter their valid credentials to register to the system.

On signing up the user is redirected to the Home screen of the system.

➤ Elements:

- Input box, to enter their valid credential data. This information's are used for registration with the system.
- Sign up button to transact the data with Firebase Authentication and Firebase database service.
- Sign in button for navigate to Sign in screen if the user already has registered.

○ Dash board screen:



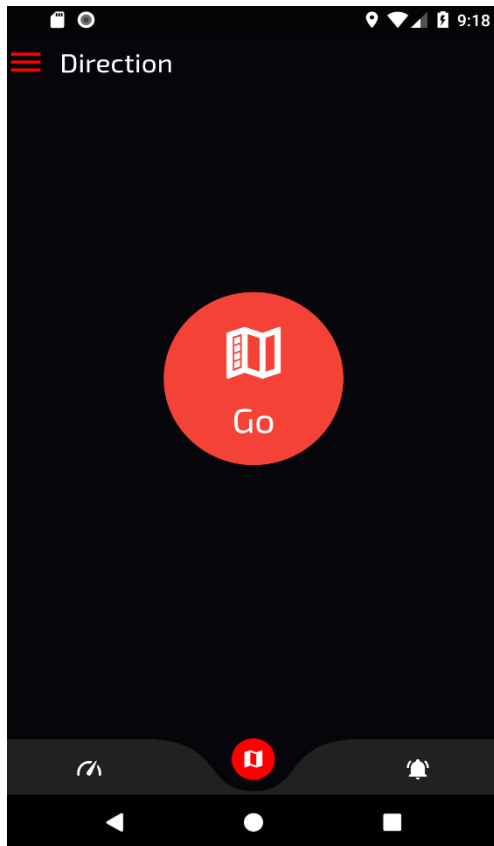
➤ Description:

Dash Board screen is the main screen where the user will get information of there vehicle like Distance to empty, Fuel, this data's are dynamic with the vehicle movement the data's change.

➤ Elements:

- Bottom navigation bar consisting of three buttons.
- Dash board button of navigation bar redirect to Dah Board screen.
- Direction button on navigation bar redirect to Direction screen.
- SOS button on navigation bar redirect to SOS screen.
- Menu button open side bar menu.

○ **Direction screen:**



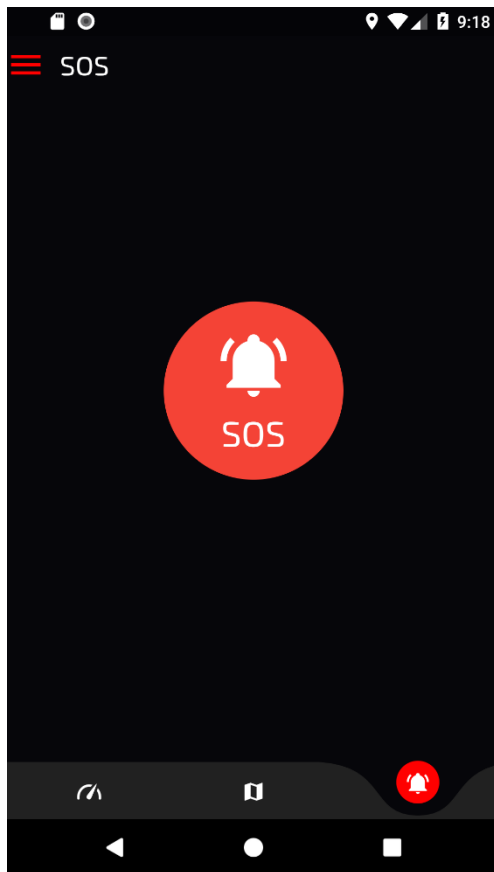
➤ **Description:**

Direction screen is use to redirect to google map for direction, this screen is one of the useful while in riding. It make it really easy to open map on the go.

➤ **Elements:**

- Bottom navigation bar consisting of three buttons.
- Dash board button of navigation bar redirect to Dah Board screen.
- Direction button on navigation bar redirect to Direction screen.
- SOS button on navigation bar redirect to SOS screen.
- Menu button open side bar menu.

○ SOS screen:



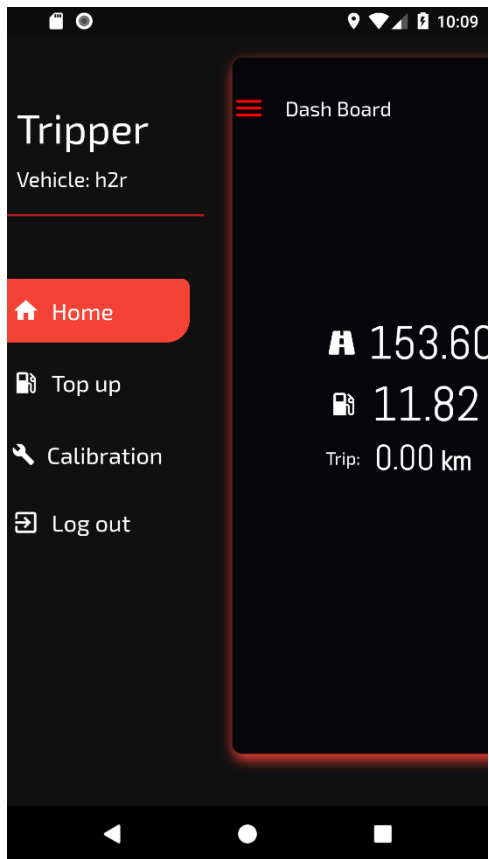
➤ Description:

SOS screen invoke SMS service for sending emergency text to a register number.

➤ Elements:

- Bottom navigation bar consisting of three buttons.
- Dash board button of navigation bar redirect to Dah Board screen.
- Direction button on navigation bar redirect to Direction screen.
- SOS button on navigation bar redirect to SOS screen.
- Main SOS button invoke SMS service to send emergency message for a situation.
- Text message which is dynamic which says about delivery status.
- Menu button to open side bar menu.

▪ Menu screen:



➤ Description:

Side menu bar consist of all redirection buttons which redirect to different screens.

➤ Elements:

- This consist of name of the app and vehicle name.
- Home button redirect to Home screen which consist of Dash board, Direction and SOS screen.
- Top up button redirects to Top Up screen.
- Calibration button redirect to Calibration screen.

Coding

Standardization of the coding:

While coding standardization that are followed are:

- **Placing comments in the code:**

Well description of a method and variables are describe using document comment “///” and comments “//”.

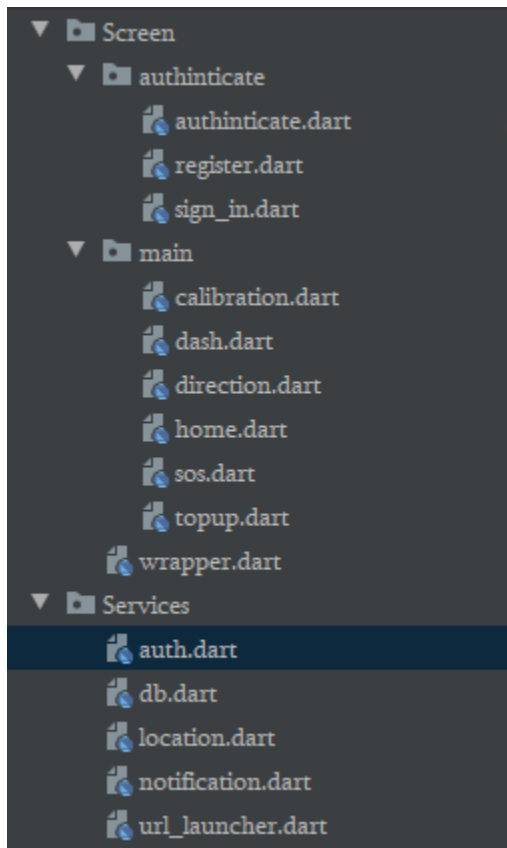
```
//fetching location from LocationService
void getLoc()
{
    ///This is to keep track of new location that the device is in.
    ///This is basically fetching data form LocationService, to keep track of
    ///current user location and the location is refreshed after five second
    ///interval
    print("test");
    location.currentLoc();
    Timer.periodic(Duration(seconds: 5), (t){
        print("new const loc -----${location.lat}");

        //adding location to location list
        if (location.lat == null && location.long == null)
        {
            print("null contPos");
        }
        else
        {
            locList.add([location.lat, location.long]); //storing in loclist
            print("array list: $locList");

            //calling locationDistance
            locationDistance(locList, snapshot, db);
        }
    });
}
```

- **Group code into logical groups:**

All logically related methods are placed in separate module for better code readability and management.



- **Common naming conventions across the code:**

Same coding conventions is followed to name the variables, functions etc. across the code. This makes the code uniform. Also, meaningful names to variables are maintained which enhances the readability of the code.

Common naming conventions are Camel case and Underscore. In Camel case, the first letter of each word is capitalized except the first word, while in Underscore naming convention you place an underscore between each word.

```
int currentIndex = 0;
```

This variable name is defined using Camel case convention.

```
void notificationActivity()
```

This method is also defined using Camel case convention,

- **Avoid too much deep nesting:**

Too much deep nesting makes the code difficult to understand so it must be avoided. While developing Tripper, too much of nesting was avoided for code maintainability kept in mind.

- **Shorter lines:**

The length of lines must not be too long because it reduces the readability. Ideally, a line must not exceed 80 characters. Here the code is kept simple and in short, while the code was exiting 80 or 90 characters it was word rapped to next line for better readability.

```
//register with email & pwd
//also pass values to the database
Future registerWithEmailAndPassword(String email, String pwd, String model,
    double milage, double fuel, String emergencyNo) async
{
    try
    {
        AuthResult res = await _auth.createUserWithEmailAndPassword(email: email,
            password: pwd);
        FirebaseUser user = res.user;
        print("auth.dart -> register(): user: ${user}, "
            "" ""
            email: $email,
            pwd: $pwd,
            model: $model,
            milage: $milage,
            fuel: $fuel
            "" ""
    )
    }
}
```

Here register with email and password method is maintained with short liner.

- **Portability:**

The programs does not contain hard-coded values for the file path, username, hostnames and the like. Instead, these is configured through parameters at the time of installation. This makes the code portable as the application can be installed on multiple systems without any changes required in the code.

Comments and Description:

○ main.dart

```
void main() => runApp(MyApp());
```

```
class MyApp extends StatelessWidget {  
  // This widget is the root of your application.  
  @override  
  Widget build(BuildContext context) {  
    ///streamProvider is providing the value from the  
    ///stream of AuthServices, which is being passed  
    ///in the value of stream provider which is of type  
    ///User because its listening to user change  
    ///and with the help of context we can use  
    ///the data of streamProvider to any widget bellow it,  
  
    /// note: a stream actively listening of data passed  
    ///    may be the data is present or not.  
    return StreamProvider<User>.value(  
      value: AuthServices().user,  
      child: MaterialApp(  
        debugShowCheckedModeBanner: false,  
        title: "Tripper",  
        theme: ThemeData(fontFamily: 'fontBody'),  
        home: Wrapper(),  
      ),  
    );  
  }  
}
```

- **Screen/main/home.dart**

This are variable declared in home.dart

```
//data
String model;
int currentIndex = 0;
double trip = 0;
var distanceToEmpty, fuel, mileage;
var locList = [];

//data snapshot from db
DataSnapshot snapshot;

//menu collapsed
bool isCollapsed = true;

//Animation objects
AnimationController _animationController;
Animation<double> _scaleAnimation;

//objects
LocationService location = LocationService();
NotificationService notifcation;
final AuthServices _auth = AuthServices();
var db;

//screen size detail
double screenWidth, screenHeight;
```

This method is used for fetching displacemetns and add to make distance covered by the user.

```
//finding Distance
void locationDistance(List a, DataSnapshot snap, DbService db) {
    print(a);
    location.distacne(a); //passing array

    var distraveled = (location.distanceTravelled / 1000).toStringAsFixed(3);
;
    print("distaceToEmpty form db: $distanceToEmpty");

    ///cannot declare distanceToEmpty over here because
    ///stream is continually sending the data and every time its
    ///getting executed and there are limit less subtraction is
    ///happening

    /// This function is to keep track of movement and fetch distance travelled
    /// while calculating trip, Distance to empty and remaining fuel. And at the
    /// same time it update Distance to empty and fuel with Firebase database

    if (distanceToEmpty != null && this.mounted) {
        print(distanceToEmpty);
        setState() {
            distanceToEmpty = distanceToEmpty - double.parse(distraveled);
            fuel = distanceToEmpty/mileage;
            trip = trip + double.parse(distraveled);
            print("total dis: $trip");
        });
        print("fuel: $fuel");
        db.curretnUpdate(distanceToEmpty,fuel);
    }
}
```

This method fetch location data from location service and convert longitude and latitude data.

```
//fetching location from LocationService
void getLoc()
{
    ///This is to keep track of new location that the device is in.
    ///This is basically fetching data from LocationService, to keep track of
    ///current user location and the location is refreshed after five seconds
    ///interval
    print("test");
    location.currentLoc();
    Timer.periodic(Duration(seconds: 5), (t){
        print("new const loc -----${location.lat}");

        //adding location to location list
        if (location.lat == null && location.long == null)
        {
            print("null constPos");
        }
        else
        {
            locList.add([location.lat, location.long]); //storing in locList
            print("array list: $locList");

            //calling locationDistance
            locationDistance(locList, snapshot, db);
        }
    });
}
```

- **Screen/wrapper.dart**

Methods declared and their description

```
// wrapper is use for providing data security so that the authenticated user
//should get about their own data.

class Wrapper extends StatelessWidget {
  @override
  Widget build(BuildContext context) {

    //accessing the streamProvider user change data
    //while the user will signing or sign out the "_auth.onAuthStateChanged"
    //will provide the user data i.e null or the user obj
    final user = Provider.of<User>(context);
    print("wrapper.dart: user- $user");

    //return home or authenticate widget
    if(user==null)
    {
      return Authenticate();
    }
    else
    {
      return Home();
    }
  }
}
```

- **Screen/authinticate/register.dart**

Declaration of text input in registration form.

```
//email
SizedBox(height: 20.0),
TextFormField(
  decoration: InputDecoration.copyWith ( hintText: "Email"),

  validator: (val) => val.isEmpty ? "Enter a valid email" : null,
  onChanged: (val){
    setState() {
      email = val;
    };
  }),

//password
SizedBox(height: 20.0),
TextFormField(
  decoration: InputDecoration.copyWith( hintText: "Password"),
  validator: (val) => val.isEmpty && val.length < 6 ?
    "Enter password greater then 6 digit" : null,
  obscureText: true,
  onChanged: (val){
    setState() {
      pwd = val;
    };
  },
  style: TextStyle(
    color: Colors.white,
  )
),
```

- **Services/auth.dart**

Streams provide an asynchronous sequence of data. This stream is defined in auth.dart this is used to get the auth change in the data base that is whether the user id is present or not.

```
//getting steam on user auth change from fireabase
```

```
Stream <User> get user
```

```
{
```

```
  /// getting user object as every time the user signin or
```

```
  /// singout and pass in the stream which notify with
```

```
  /// the help of contex to any widget liying bellow
```

```
  /// this widget. (user val is null or an user onj)
```

```
return _auth.onAuthStateChanged
```

```
  .map((FirebaseUser user) => _userForomFirebaseUser(user) );
```

```
}
```

The bellow method is used for registration of user using firebase

```
//register with emial & pwd
//also pasign values to the database
Future registerWithEmailAndPassword(String email, String pwd, String
model, double milage, double fuel, String emergencyNo) async
{
  try
  {
    AuthResult res = await _auth.createUserWithEmailAndPassword(email:
      email, password: pwd);
    FirebaseUser user = res.user;
    print("auth.dart -> register(): user: ${user}),"
      """"
      email: $email,
      pwd: $pwd,
      model: $model,
      milage: $milage,
      fuel: $fuel
      """"
    );
    DbService(uid: user.uid).createData(email, pwd, model, milage, fuel,
      emergencyNo);
    print("auth.dart: User created successfully");
    return _userForomFirebaseUser(user);
  }
  catch(e)
  {
    print("auth.dart:");
    print("catch regwith em and pwd:${e.toString()}");
    return null;
  }
}
```

This bellow method is used for signing in using email and password.

```
//Sign in with email and pwd
Future signinWithEmailAndPassword(String email,String pwd) async
{
  try
  {
    AuthResult res = await _auth.signInWithEmailAndPassword(email: email,
      password: pwd);
    FirebaseUser user = res.user;
    return _userForomFirebaseUser(user);
  }
  catch(e)
  {
    print("auth.dart:");
    print("catch reg with em and pwd:${e.toString()}");
    return null;
  }
}
```

This method is use for logging out from the system.

```
// sign out
```

```
Future signOut() async
{
  try
  {
    return await _auth.signOut();
  }
  catch(e)
  {
    print("auth.dart: catch error form signout ${e.toString()}");
    return null;
  }
}
```

- **Services/location.dart**

This bellow method is used for getting location continually and also it is capable of calculating the

```
//getting continues location
void currentLoc()
{
  ///This method fetch the location information
  ///of the device and also it fetch information
  ///in background.
  BackgroundLocation.startLocationService();
  BackgroundLocation.checkPermissions().then((status) {
    print("status");
    print(status);
  });

  BackgroundLocation.getLocationUpdates((loc) {
    print("${loc}");
    lat = loc.latitude;
    long = loc.longitude;
    print("=====bgloc from
      location.dart=====");
  });
}
```

This method is used for calculating total distance travelled.

```
//calculating displacement of every movent
Future<void> distacne(List a) async
{
    ///This basically calculate the displacement
    ///later on this displacements are added to find
    ///total distance travelled
    print(a);
    if(a.length > 1)
    {
        for (var i = 0; i<a.length-1; i++)
        {
            print("array length: ${a.length}");
            print("index $i val->${a[i]}");

            var distanceBetweenPoints = SphericalUtil.computeDistanceBetween(
                LatLng(a[i][0], a[i][1]),
                LatLng(a[i+1][0], a[i+1][1])
            );

            distanceTravelled = distanceBetweenPoints; //taking upto 2 decimal
        }

        Future.delayed(const Duration(seconds: 10), () {
            print("cleare executed");
            a.clear();
        });
    }
    else
    {
        if(distanceTravelled == null)
            distanceTravelled = 0;
    }
}
```

Validation checks:

Validation and verification are two ways to check that the data entered into a computer is correct. Data entered incorrectly is of little use.

○ **Format check:**

Format check is a validation check, which checks the data is in the right format or not.

While Tripper was developed every text field has sets of validation where the format check was also taken care of. In this app the user virtual keyboard is restricted with only numbers in field where only number should be enter.

```
//emergency no
SizedBox(height: 20.0),
TextFormField(
  decoration: InputDecoration.copyWith( hintText: "Emergency
    phone number"),
  keyboardType: TextInputType.number,
  validator: (val) => val.isEmpty && val.length < 10 && val.length > 10 ?
    "Enter a valid phone no" : null,
  onChanged: (val)
    setState(() {
      emergencyNo = val;
    });
  },
  style: TextStyle(
    color: Colors.white,
  ),
),
```

Here the keyboard is restricted with numeric input only, this is achieved using validator property of the text field.

- **Length check:**

It checks the data isn't too short or too long.

While Tripper is developed it was keep in mind that the fields should not be too long or too short according to the requirement of the data in a text input field.

```
//emergency no
SizedBox(height: 20.0),
TextFormField(
  decoration: textInputDecoration.copyWith( hintText: "Emergency phone
    number"),
  keyboardType: TextInputType.number,
  validator: (val) => val.isEmpty && val.length < 10 && val.length > 10 ?
    "Enter a valid phone no" : null,
  onChanged: (val){
    setState() {
      emergencyNo = val;
    });
  },
  style: TextStyle(
    color: Colors.white,
  ),
),
```

Here the length check is done where the input data should not be less than 10 or greater than 10, this is achieved using validator property of the text field.

○ Presence check:

Checks that data has been entered into a field.

Presence check is enabled in Tripper where the text fields cant be empty. If a user keep a field empty then it shows a request message saying “fields can’t be empty”.

```
//emergency no

    SizedBox(height: 20.0),
    TextFormField(
      decoration: InputDecoration.copyWith( hintText: "Emergency phone
        number"),
      keyboardType: TextInputType.number,
      validator: (val) => val.isEmpty && val.length < 10 && val.length > 10 ?
        "Enter a valid phone no" : null,
      onChanged: (val){
        setState() {
          emergencyNo = val;
        });
      },
      style: TextStyle(
        color: Colors.white,
      ),
    ),
  ),


```

This validation check is achieved using validation property in text field. Here it checks if the field is empty.

Testing

Mobile application testing is a procedure to test mobile applications for usability, functional, and consistency glitches.

Functional Testing Test Cases:

The functional testing of Mobiles normally consists in the areas of testing user interactions as well as testing the transactions. The various factors which are relevant in functional testing done in Tripper are

- The required mandatory fields are working as required.
- The mandatory fields are displayed in the screen in a distinctive way than the non-mandatory fields.
- Application is works as per as requirement whenever the application starts/stops.
- Application goes into minimized mode whenever there is an incoming phone call. In order to validate the same, we need to use a second phone, to call the device.
- The phone is able to store, process and receive SMS whenever the app is running. In order to validate the same, we need to use a second phone to send SMS to the device which is being tested and where the application under test is currently running.
- The device is able to perform required multitasking requirements whenever it is necessary to do so.
- Scrolling scenarios are being enabled in the application as necessary.
- The navigation between relevant modules in the application are as per the requirement.
- The installed application enables other applications to perform satisfactorily, and it does not eat into the memory of the other applications.
- The application resumes at the last operation in case of a hard reboot or system crash.

- The installation of the application can be done smoothly provided the user has the necessary resources and it does not lead to any significant errors.
- The application performs according to the requirement in all versions of Mobile that is 2g, 3g and 4g.
- The application provides an available user guide for those who are not familiar to the app

Performance Testing Test Cases

This type of testing's fundamental objective is to ensure that the application performs acceptably under certain performance requirements such as access by a huge number of users or the removal of a key infrastructure part like a database server.

The general test scenarios for Performance Testing done are:

- The application performs as per the requirement under different load conditions.
- The current network coverage is able to support the application at peak, average and minimum user levels.
- The existing client-server configuration setup provides the required optimum performance level.
- The response time of the application is as per as the requirements.
- Android SKD 21 that aka Android 5.0 is the minimum requirement of this application to run in optimal performance with 1 GB of ram and 2.7 GHz processor with minimum of dual core.
- This application run in background and while travelling it is recommended to use a charger because it depends on how long a ride or a drive can be.
- There is no interruption when network is changed to WIFI from 2G/3G or vice versa.
- The battery consumption in as required.
- The application run well while there is interruption in internet, as Firebase support offline data sync.

Security Testing Test Cases

The fundamental objective of security testing is to ensure that the application's data and networking security requirements are met as per guidelines.

The following are the most crucial areas of checking that's has been done are:

- The application is not permitting an attacker to access sensitive content or functionality without proper authentication.
- The application has a strong password protection system and it does not permit an attacker to obtain, change or recover another user's password.
- It is maintained with dynamic dependencies and take measures to prevent any attacker for accessing these vulnerabilities.
- To protect the application and the network from the denial of service attacks.
- Data storage and data validation requirements are maintained.
- Tripper is well protecting against malicious client side injections.
- It is well protecting against malicious runtime injections.
- Regular audits will be done for data protection analysis.
- Different data streams is analyzed and preventing any vulnerabilities from these.

Usability Testing Test Cases

The usability testing process of the Mobile application is performed to have a quick and easy step application with less functionality than a slow and difficult application with many features. The main objective is to ensure that we end up having an easy-to-use, intuitive and similar to industry-accepted interfaces which are widely used.

- The buttons have the required size and be suitable to big fingers.
- The buttons are placed in the same section of the screen to avoid confusion to the end users.
- The icons are natural and consistent with the application.
- To ensure that the application provides a method for going back or undoing an action, on touching the wrong item, within an acceptable duration.
- The contextual menus are not overloaded because it has to be used quickly.
- The text is kept simple and clear to be visible to the users.
- The font size is big enough to be readable and not too big or too small.
- All strings are converted into appropriate languages whenever a language translation facility is available.
- The application items are always synchronized according to the user actions.

Compatibility Testing Test Cases

Compatibility testing on mobile devices is performed to ensure that since mobile devices have different size, resolution, screen, version and hardware so the application should be tested across all the devices to ensure that the application works as desired.

The following are the most prominent areas for compatibility testing.

- The user Interface of the application is as per the screen size of the device, no text/control is partially invisible or inaccessible.
- The text is readable for all users for the application.
- The application is running. The application is minimized or suspended on the event of a call and then whenever the call stops the application is resumed.

System Security measures

These Measures apply to anyone who accesses, uses, or controls this app, including, but not limited to faculty, administrators, staff, students, those working on behalf of the University, guests, tenants, contractors, consultants, visitors and/or individuals authorized by affiliated institutions and organizations.

Database/data security:

Database security testing is done to find the loopholes in security mechanisms and about finding the vulnerabilities or weaknesses of database system.

The main target of database security testing is to find out vulnerabilities in a system and to determine whether its data and resources are protected from potential intruders. Security testing defines a way to identify potential vulnerabilities effectively, when performed regularly.

○ Unauthorized Access to data:

Another type of attack is gaining unauthorized access to data within an application or database system. This includes the unauthorized user to access the database without login procedure and steal any others information.

Tripper is secure form unauthorized access of data, as Google Firebase authentication service is used for authenticity of a user it is one of the most secure way implemented on this app. Here the authenticate users are only allow to access their data. The register user have to login with their valid credentials to get a access.

- **Password Cracking:**

This is the most important check while performing database system testing. To access critical information, hackers can use a password-cracking tool or can guess a common username/password. These common passwords are easily available on internet and password cracking tools exist freely.

As the authentication of a user is done using Firebase authentication service, google provide password security that is the password is encrypted by Google's encryption and no one can see the password in the console. That make the password highly impossible to cracking.

- **Security Audit of Database System:**

A security audit is a process of evaluating company's security policies at a regular time interval to determine whether necessary standards are followed or not. Various security standards can be followed as per business requirement to define the security policy and then assessment of set policies against those standards can be done.

A security audit of database system will be done frequently to prevent any flaw in the security measure of the system.

- **Penetration Testing:**

A penetration test is an attack on a computer system with the intention of finding security loopholes, potentially gaining access to it, its functionality and data.

- **Risk Finding:**

Risk Finding is a process of assessing and deciding on the risk involved with the type of loss and the possibility of vulnerability occurrence. This is determined within the organization by various interviews, discussions and analysis.

This app is stable in case of any kind of security risk, but there is more to be taken care of in future.

- **Minimize Value of Your Database:**

Attackers can only get their hands on what is stored in a database, so there is no confidential information that doesn't need to be there. And actively managing of the data will be done so that unnecessary information that can be removed. Data that must be retained for compliance or other purposes can be moved to more secure storage – perhaps offline -- which is less susceptible to database security threats.

- **Database Firewalls:**

Your database server should be protected from database security threats by a firewall, which denies access to traffic by default. The only traffic allowed through should come from specific application or web servers that need to access the data. The firewall should also protect your database from initiating outbound connections unless there is a specific need to do so.

Data security in firebase is done using Firebase rules, which prevent unnecessary access of the database from unauthorized access.

- **Creation of User profiles and access rights:**

User permissions and access rights are specified in profiles and permission sets. To use them effectively, understand the differences between profiles and permission sets.

IF the user doesn't have any account, he/she has to create an account to be an authorized member of the system and to access the elements and benefits from the system, every user is assigned only one profiles to assign the minimum permissions and access settings for specific groups of users. Then use permission sets to grant more permissions as needed.

Once the profile is created user can access all the needed element of the system.

Future scope

This system is developed for the travelers and daily commuters, and there need doesn't end with this current phase of the system. So, this application will still be developed in future to make it more efficient for more accurate data and much more to be added information.

Upcoming features are:

- **Weather module:**

This is a upcoming module. The responsibility of this module is to show the weather forecast. This will also show future forecast of the weather so that a user can take their decision before going out.

- **Speed module:**

This will be responsible for presenting the current speed of the vehicle. This will also inform about the economy speed of the vehicle and will put a warning if the vehicle is running above 100 km/h.

- **Easy connect vehicle:**

This module will be enabled with Bluetooth information fetching system. This app in future will be capable enough to connect to any vehicle and fetch vehicle information to make more precise calculation and presenting a precise information.

- **Login module improvement:**

Login module will be enabled with forgot password system in future so that if the user forgets their password, they can recover.

- **Login user verification:**

User verification will be enabled which will make sure that a genuine user register to the system.

- **iOS development betterment:**

The iOS modules will be develop in a proper manner.

Further development:

The development of this app does not end over here. This app will be in throw check of security and in near future the iOS module will improved. Also, the security rule implementation will be done in more strong manner to Firebase database rules. This app will be developed for web and for PC use also, which is currently runnable in Android, iOS beta.

Assumptions

An assumption is something that is believed to be true. It's an event that you can expect to happen during a project. However, that certainty isn't supported by factual proof, it comes from experience. Just like dependencies and constraints, assumptions are events that are outside of the project manager's and team's control. But unlike constraints, which put restrictions on a project and can pose a danger to its successful completion, assumptions open possibilities for it and make it possible for the project to finish successfully. This is how you can differentiate assumptions from constraints and dependencies.

Operating System:

Minimum version assumed Android 5.0 and iOS 9.

Hardware config:

Minimum hardware configuration assumed to be

Android: minimum of 1GB RAM, 1.2 GHz dual core processor.

iOS: minimum of 1GB RAM, 1.2 GHz dual core processor.

Device screen size:

Assumption of the device screen size is range from 4 inch to 6.5 inch.

GPS:

Device has good GPS in it.

Battery:

Device battery health is in good condition.

Glossary

Flutter:

Flutter is an open-source UI software development kit created by Google. It is used to develop applications for Android, iOS, Windows, Mac, Linux, Google Fuchsia and the web.

The first version of Flutter was known as codename "Sky" and ran on the Android operating system. It was unveiled at the 2015 Dart developer summit, with the stated intent of being able to render consistently at 120 frames per second. During the keynote of Google Developer Days in Shanghai, Google announced Flutter Release Preview 2 which is the last big release before Flutter 1.0. On December 4, 2018, Flutter 1.0 was released at the Flutter Live event, denoting the first "stable" version of the Framework. On December 11, 2019, Flutter 1.12 was released at the Flutter Interactive event.

Dart platform:

Flutter apps are written in the Dart language and make use of many of the language's more advanced features.

On Windows, macOS and Linux via the semi-official Flutter Desktop Embedding project, Flutter runs in the Dart virtual machine which features a just-in-time execution engine. While writing and debugging an app, Flutter uses Just in Time compilation, allowing for "hot reload", with which modifications to source files can be injected into a running application. Flutter extends this with support for stateful hot reload, where in most cases changes to source code can be reflected immediately in the running app without requiring a restart or any loss of state. This feature as implemented in Flutter has received widespread praise.

Firestore:

Firestore gives you the tools to develop high-quality apps, grow your user base, and earn more money. We cover the essentials so you can monetize your business and focus on your users.

○ **Firestore Authentication:**

Firestore Authentication provides backend services, easy-to-use SDKs, and ready-made UI libraries to authenticate users to your app. It supports authentication using passwords, phone numbers, popular federated identity providers like Google, Facebook and Twitter, and more.

Firestore Authentication integrates tightly with other Firestore services, and it leverages industry standards like OAuth 2.0 and OpenID Connect, so it can be easily integrated with your custom backend.

○ **Firestore Realtime Database:**

Store and sync data with our NoSQL cloud database. Data is synced across all clients in realtime, and remains available when your app goes offline.

The Firestore Realtime Database is a cloud-hosted database. Data is stored as JSON and synchronized in realtime to every connected client. When you build cross-platform apps with our iOS, Android, and JavaScript SDKs, all of your clients share one Realtime Database instance and automatically receive updates with the newest data.

Android:

Android is a mobile operating system based on a modified version of the Linux kernel and other open source software, designed primarily for touchscreen mobile devices such as smartphones and tablets. Android is developed by a consortium of developers known as the Open Handset Alliance, with the main contributor and commercial marketer being Google.

iOS:

iOS is a mobile operating system created and developed by Apple Inc. exclusively for its hardware. It is the operating system that presently powers many of the company's mobile devices, including the iPhone, and iPod Touch; it also powered the iPad prior to the introduction of iPad OS in 2019. It is the second most popular mobile operating system globally after Android.

Bibliography

- <https://stackoverflow.com/>
- <https://www.youtube.com/>
- <https://flutter.dev/>
- <https://medium.com/>
- <https://pub.dev/>
- <https://fonts.google.com/>
- <https://api.flutter.dev/flutter/material/Icons-class.html>